



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A04 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 17 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN3

ENUNCIADO.....4

Probad todo el código del Apartado 44

Caso práctico 4.....5

 Módulos5

Ponte a prueba 4.....6

 Importación de librerías externas6

Claves de resolución:7

DESARROLLO DE LA PRACTICA.....8

Descripción8

Contenido del archivo8

 • Código del Apartado 4 de teoría8

 • Caso Práctico 4 – Módulos8

 • Ponte a Prueba 4 – Importación de librerías externas9

Cómo visualizarlo.....11

 Requisitos11

 Pasos11

Objetivos de la práctica.....11

 • Probar y comprender (o intentarlo al menos) el código del material teórico del Apartado 4. ...11

 • Crear y exportar módulos JavaScript aprovechables o reutilizables.11

 • Importar definiciones desde otros archivos usando import.....11

 • Utilizar librerías externas, en este caso Chart.js, mediante el sistema de módulos explicado en el ejercicio, ES6.....11

 • Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.11

 • Presentar el ejercicio en formato PDF junto con los archivos fuente y pantallazos, tanto de implementación (captura del VS Code) como de ejecución (capturas en consola o página del navegador).11

CONCLUSIÓN12

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 4” deberemos incluir en uno o varios archivos (se recomienda en mi opinión en varios) o scripts todo el código que se muestre en este apartado de la teoría, así como su ejecución y autoría con el nombre del autor y fecha.

En el “Caso Práctico 4” vamos a crear un módulo en un archivo con extensión .js y almacenaremos el código JavaScript que realizamos en el Caso Práctico 3. Tras ello, exportaremos las definiciones para poder utilizarlas mediante import dejando únicamente las definiciones de los prototipos.

En el “Ponte a Prueba 4” a través del HTML proporcionado utilizaremos una librería externa para mostrar una gráfica. Asimismo, modificaremos el código del archivo para importar la librería mediante ES6 y que pueda funcionar.

ENUNCIADO

Probad todo el código del Apartado 4

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWECE_U02_A04_principal.html

```
<> DWECE_U02_A04_z_PaP.html <> DWECE_U02_A04_principal.html x
<> DWECE_U02_A04_principal.html > {} "DWECE_U02_A04_principal.html" > html > body > script
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6      <head>
7          <meta charset="UTF-8">
8          <meta name="viewport" content="width=device-width, initial-scale=1.0">
9          <title>4.3. Módulos en HTML</title>
10     </head>
11     <body>
12         <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14         <script type="module">
15             import * as miLibreria from './miLibreria.js';
16
17             miLibreria.saludar("Juan");
18             miLibreria.despedir("Juan");
19         </script>
20
21         <!-- Prueba de Autoría -->
22         <p>Autor: Mario Segura Abad<br></p>
23         <p>Fecha de realización: 16/10/2025</p>
24     </body>
25 </html>
```

Captura de pantalla – miLibreria.js

```
JS miLibreria.js x
JS miLibreria.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 16/10/2025
3
4
5  // 4.2. Import y expot
6
7  // Fichero miLibreria.js
8  function saludar(usuario) {
9      console.log(`Hola, ${usuario}`);
10 }
11
12 function secreto() {
13     console.log("Sssss... es un secreto...");
14 }
15
16 function despedir(usuario) {
17     secreto();
18     console.log(`Adiós, ${usuario}`);
19 }
20 // Exportación de objetos.
21 // Se exporta un objeto con dos propiedades.
22 // Cada propiedad hace referencia a una de las funciones definidas arriba.
23 // La función 'secreto' no se exporta, por lo que no es visible fuera del módulo;
24 // sin embargo, dentro del módulo sí que puede utilizarse.
25 export {
26     saludar,
27     despedir
28 }
29 }
```

Captura de pantalla – principal.js

```
JS principal.js ×
JS principal.js
1 // Autor: Mario Segura Abad
2 // Fecha: 16/10/2025
3
4
5 // 4.2. Import y export
6
7 // Fichero principal.js
8
9 // Importación de las funciones exportadas por el módulo
10 // Mediante esta sintaxis se pueden indicar las funciones que se deseen importar,
11 // que no tienen por qué ser todas
12 import { saludar, despedir } from './miLibreria';
13
14 // A partir de aquí se pueden utilizar las funciones importadas, 'saludar' y 'despedir'
15 saludar("Juan"); // Hola, Juan
16 despedir(); // "Sssssh... es un secreto..."
17 // Adiós, Juan
18
19 // Fichero principal.js
20
21 import * as miLibreria from './miLibreria.js';
22 // El objeto 'miLibreria' tiene definidas todas las funciones exportadas por la librería
23
24 miLibreria.saludar("Juan"); // Hola, Juan
25 miLibreria.despedir(); // Sssssh... es un secreto...
26 // Adiós, Juan
```

Caso práctico 4

Módulos

Crea un módulo denominado en un archivo con extensión **.js** y almacena el código JavaScript creado en el Caso práctico 3. Deja únicamente las definiciones de los prototipos y expórtalas para que puedan ser utilizadas mediante **import**.

Ejemplo de implementación:

```
JS enemigos.js ×
CasoPractico4 > JS enemigos.js > Enemigo
1 // Autor: Mario Segura Abad
2 // Fecha: 16/10/2025
3
4
5 // -----
6 // Definición de clases
7 // -----
8
9 export class Enemigo {
10   constructor(energia = 10) {
11     this.energia = energia;
12   }
13
14   mover() {
15     console.log(`Propiedades de Enemigo: ${this.energia}`);
16   }
17 }
18
19 // Clase derivada (Orco) que hereda de Enemigo
20 export class Orco extends Enemigo {
21   constructor(energia = 10, fuerza = 15) {
22     super(energia); // Llama al constructor de Enemigo
23     this.fuerza = fuerza;
24   }
25
26   disparar() {
27     console.log(`Propiedad de Orco: ${this.fuerza}`);
28   }
29 }
30
```

A continuación, crea una página HTML que contenga un *script* que importe dicho módulo y cree un enemigo de cada tipo utilizando las funciones importadas.

Ejemplo de implementación:

```
<> index.html x
CasoPractico4 > <> index.html > { } "index.html" > html > body > script
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 16/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="UTF-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1.0">
9      <title>Caso práctico 4 - Módulos</title>
10   </head>
11   <body>
12     <h1>Abrir la consola (F12) para ver el resultado</h1>
13
14     <script type="module">
15       import { Enemigo, Orco, Troll } from './enemigos.js';
16
17       // Crear objetos de cada clase
18       const enemigo = new Enemigo(10);
19       const orco = new Orco(10, 15);
20       const troll = new Troll(10, 20);
21
22       // Probar métodos
23       enemigo.mover(); // Propiedad de Enemigo: 10
24       orco.mover(); // Propiedad de Enemigo: 10
25       orco.disparar(); // Propiedad de Orco: 15
26       troll.mover(); // Propiedad de Enemigo: 10
27       troll.golpear(); // Propiedad de Troll: 20
28     </script>
29
30     <!-- Prueba de Autoría -->
31     <p>Autor: Mario Segura Abad<br></p>
32     <p>Fecha de realización: 16/10/2025</p>
33   </body>
34 </html>
```

Ponte a prueba 4

Importación de librerías externas

Se proporciona un archivo HTML que utiliza una librería externa para mostrar una gráfica. Dicha librería es **chart.js**. Todo el código está preparado para funcionar, salvo por un detalle: falta importar el código de dicha librería para que pueda funcionar.

Modifica el código del archivo para importar la librería mediante el sistema de módulos ES6 y que pueda funcionar la página. Está alojado en el CDN **jsdelivr**; concretamente, en la página <https://www.jsdelivr.com/package/npm/chart.js?path=dist>.

A continuación, se indica la lista de objetos que debes importar: **Chart**, **LinearScale**, **BarController**, **CategoryScale**, **BarElement**.

Ejemplo de implementación:

```
<> DWEC_U02_A04_z_PaP.html X
PonteAPrueba4 > <> DWEC_U02_A04_z_PaP.html > {} "DWEC_U02_A04_z_PaP.html" > html > body > div
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 17/10/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="utf-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <title>Ponte a Prueba 4 – Uso de librerías externas</title>
10   </head>
11   <body>
12     <h1>Uso de librerías externas con Chart</h1>
13     <div>
14       <canvas id="myChart" width="400" height="400"></canvas>
15     </div>
16
17     <script type="module">
18       // Importar funciones de la librería externa a través del CDN
19       import { Chart, LinearScale, BarController, CategoryScale, BarElement }
20       from 'https://cdn.jsdelivr.net/npm/chart.js@4.5.1/+esm';
21
22       Chart.register(LinearScale, BarController, CategoryScale, BarElement);
23       const ctx = document.getElementById('myChart');
24       const myChart = new Chart(ctx, {
25         type: 'bar',
26         data: {
27           labels: ['Red', 'Blue', 'Yellow', 'Green', 'Purple', 'Orange'],
28           datasets: [{
29             label: '# of Votes',
30             data: [12, 19, 3, 5, 2, 3],
31
59
60     <!-- Prueba de Autoría -->
61     <p>Autor: Mario Segura Abad<br></p>
62     <p>Fecha de realización: 17/10/2025</p>
63   </body>
64 </html>
```

Claves de resolución:

Adjunto, dispones del archivo **DWEC_U02_A04_PP_E.html**, con el código de partida.

Al acceder a la URL proporcionada, verás varios enlaces a diferentes archivos. Dado que el sistema de módulos todavía no está implantado al 100%, la mayoría de las librerías ofrecen enlaces para realizar la carga mediante el sistema tradicional de importación al objeto global.

En el caso de que proporcionen archivos compatibles con el sistema de módulos, estos tienen la extensión **.esm.js**. Por tanto, deberemos copiar la ruta al archivo **chart.esm.js**.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 2

Descripción

Este proyecto se centra en el uso de **módulos ES6** en JavaScript y la **importación de librerías externas** para mejorar el desarrollo web en entorno cliente mediante un trabajo más visual y representativo gracias a los gráficos.

Se ha probado, **como siempre**, el código de la teoría del apartado 4 de teoría y el desarrollo del propio caso práctico.

El objetivo principal ha sido intentar dividir el código en dos archivos “reutilizables” (se refiere con dicho término a poder usar dicho código en otro script completamente diferente mediante la palabra reservada `import`), importar definiciones desde módulos externos y utilizar una librería CDN para mostrar gráficos en nuestra página web.

Contenido del archivo

- **Código del Apartado 4 de teoría**

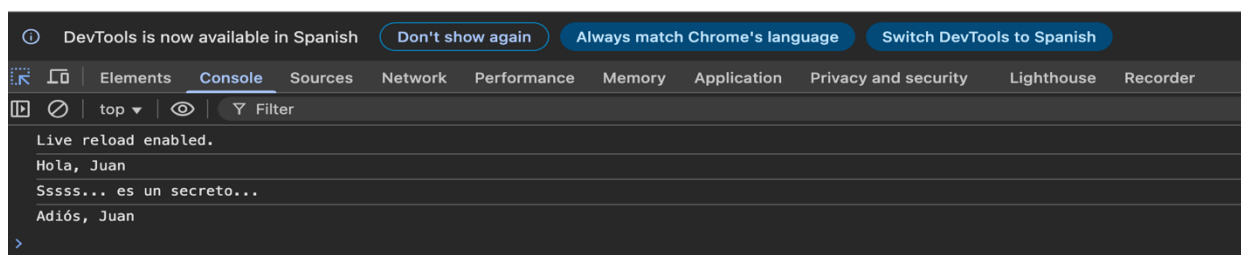
Se ha recopilado y probado absolutamente todo el código de la teoría del Apartado 4 de JavaScript con relación a la creación de módulos, exportación (mediante la palabra reservada `export`) de funciones e importación de funciones mediante `import`. Cada archivo en concreto incluye el nombre y la fecha en comentarios y en la propia página web. Se han capturado los resultados, tanto en el navegador (en pantalla) como en consola.

Ejemplo de ejecución:

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 16/10/2025



- **Caso Práctico 4 – Módulos**

1. **Creación del script enemigos.js**

Se ha extraído el código del Caso Práctico 3 y se ha convertido en un módulo que define tres tipos de enemigos: el propio `Enemigo` (clase Padre), `Troll` y `Orco` (clases hijas de `Enemigo`). Cada enemigo se define, asimismo, como una clase, y se exporta para su uso externo, pues se puede extrapolar a cualquier script mediante su importación.

2. **Página HTML que importa el módulo**

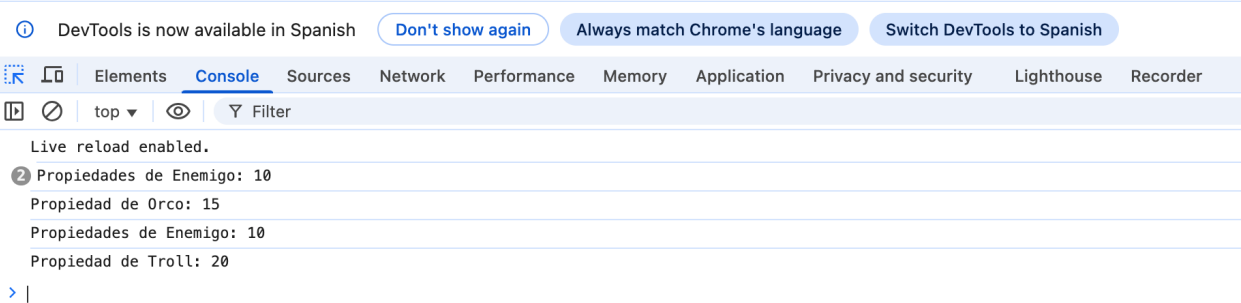
Se ha creado una página HTML que importa el módulo `enemigos.js` y crea una instancia de cada tipo de enemigo.

Ejemplo de ejecución:

Abrir la consola (F12) para ver el resultado

Autor: Mario Segura Abad

Fecha de realización: 16/10/2025

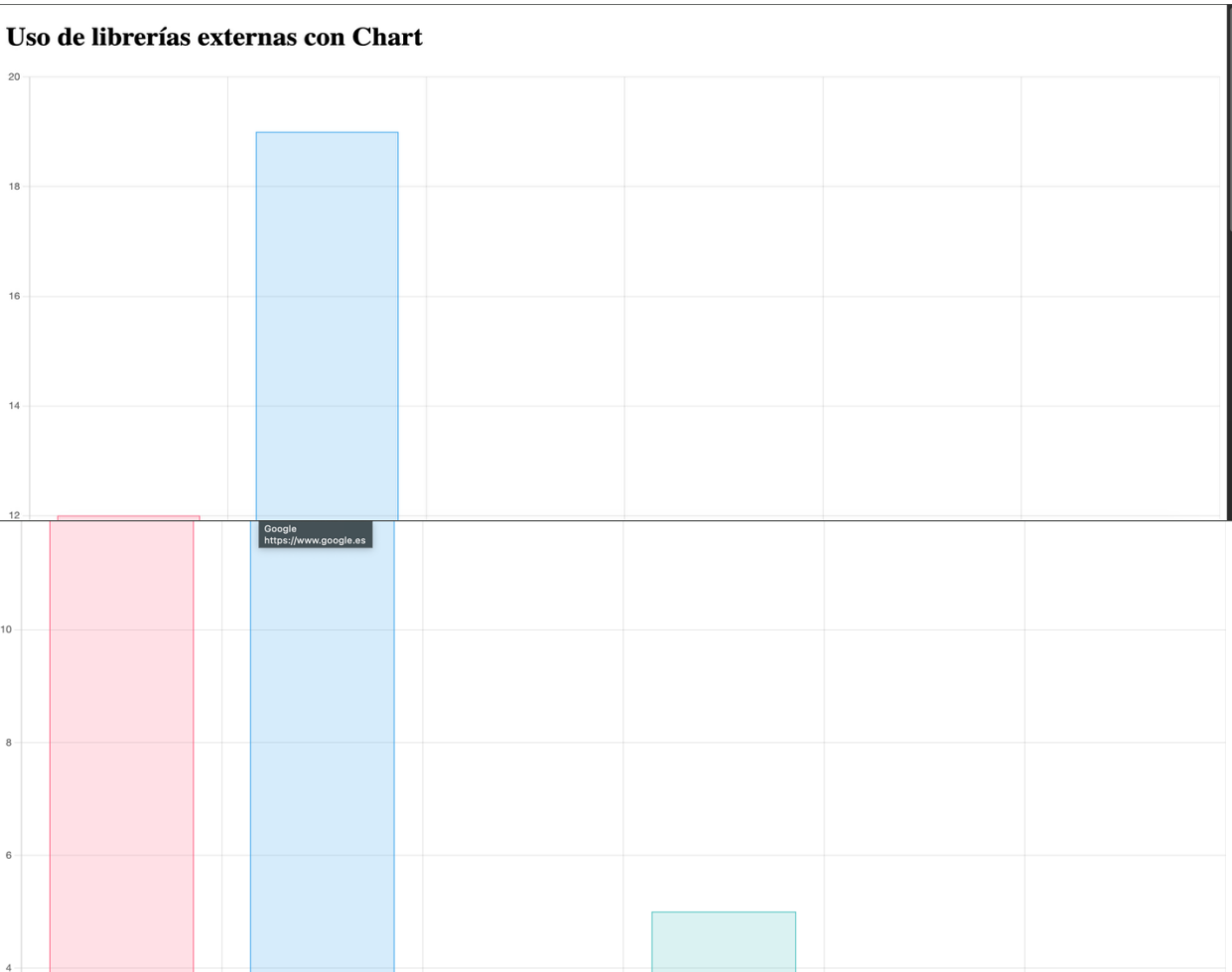


- **Ponte a Prueba 4 – Importación de librerías externas**

Modificación del archivo HTML para poder importar las librerías, en este caso, Chart.js

Se ha modificado el archivo DWEC_U02_A04_PP_E.html proporcionado en la tarea para importar correctamente la librería chart.js desde el CDN jsdelivr utilizando el formato ESM (ES6).

Ejemplo de ejecución:





Cómo visualizarlo

Requisitos

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web para scripts con `document.write()` o `prompt()`.
- Node.js o la consola del navegador (F12) para ejecutar scripts con `console.log()`.

Pasos

1. Abrir los archivos `.js` o `.html` en el editor de código.
2. Ejecutar los scripts en el navegador o consola, según el tipo.
3. Observar el resultado en pantalla o consola.
4. Revisar las capturas de ejecución incluidas en la carpeta correspondiente.

Objetivos de la práctica

- Probar y comprender (o intentarlo al menos) el código del material teórico del Apartado 4.
- Crear y exportar módulos JavaScript aprovechables o reutilizables.
- Importar definiciones desde otros archivos usando `import`.
- Utilizar librerías externas, en este caso Chart.js, mediante el sistema de módulos explicado en el ejercicio, ES6.
- Documentar correctamente cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el ejercicio en formato PDF junto con los archivos fuente y pantallazos, tanto de implementación (captura del VS Code) como de ejecución (capturas en consola o página del navegador).

CONCLUSIÓN

Esta práctica nos ha permitido aplicar el sistema de módulos ES6 en JavaScript como he comentado anteriormente, una herramienta muy útil para poder organizar y mostrar el código en futuros proyectos reales. Además, se ha comprobado cómo exportar clases y funciones (export) desde un archivo .js y cómo importarlo (import) en un archivo HTML para crear instancias de objetos y ejecutar sus métodos correspondientes. Además, se ha trabajado con una librería externa en concreto (Chart.js) para generar gráficos dinámicos con una presentación más chula, intuitiva y clara, aprendiendo a importar componentes específicos desde un CDN como jsdelivr gracias a las explicaciones pertinentes tanto en la teoría como en la explicación del ejercicio. La práctica nos ha servido para reforzar conceptos como la reutilización de código y, lo que di el año pasado, Programación Modular. Por último, se ha reforzado la importación selectiva y visualización de datos.